本章路線

這章要回答的核心問題是：作業系統應如何分層與暴露介面，讓功能可維護，又不犧牲隔離與效能？

前置知識。 mode bit、中斷與 trap、呼叫堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 追蹤 system call 從使用者函式庫到核心服務的路徑
- 比較 monolithic、layered、microkernel、module 與 hybrid 結構
- 解釋除錯、產生系統映像與開機所需資料

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 作業系統服務

- 服務可分成使用者導向與系統效率導向。程式執行、I/O、檔案、通訊與錯誤處理直接服務應用；資源配置、帳務與保護維持系統運作。
- 服務存在不代表核心都直接實作。使用者空間 daemon、函式庫與核心可合作提供同一個介面。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 服務、提供者與完成契約

作業系統服務不等於一張syscall名稱表。User interface、program execution、I/O、file manipulation、communication與error detection支援使用者工作；resource allocation、accounting、protection/security支援多個工作共存。服務可由kernel、library、daemon與應用工具合作完成，保護邊界由實際權限與物件存取決定。

複製檔案所需服務	固定實作中的提供者	能觀察的完成條件
解讀來源/目的名稱	shell與copy工具	形成兩個pathname；不是kernel解析shell語法
找檔案、檢查權限	VFS/filesystem	open成功取得descriptor或明確錯誤
傳輸資料	library API→kernel I/O	每次回傳已完成bytes；不能假定等於請求
配置/保護	kernel與filesystem	fd、cache、storage容量及存取權都合法
儲存結果	filesystem/device	檔案同步與directory entry持久性分開

2.1 服務、提供者與完成契約 (續)

User program和system program通常受相同的保護規則；「系統工具」這個名字不會自動授與kernel privilege。反過來，具管理權限的user-space daemon仍在user mode，不能把權限身分與CPU mode混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 6000-byte複製：每次read後都必須寫完

固定來源是6000 bytes且不再改動，目的為新建空regular file；blocking I/O，無其他writer，buffer4096 bytes。成功路徑刻意包含short write；所有返回值皆實際參與進度計算。

事件	本次回傳bytes	來源offset	目的offset/長度	buffer尚待寫
open來源與建立目的成功	—	0	0	0
read(4096)	4096	4096	0	4096
write(4096)	1000	4096	1000	3096
write剩餘buffer[1000:]	3096	4096	4096	0
read(4096)	1904	6000	4096	1904
write(1904)	1024	6000	5120	880
write剩餘buffer[1024:]	880	6000	6000	0
read(4096)	0，EOF	6000	6000	0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1　6000-byte複製：每次read後都必須寫完 （續）

不含directory額外同步，這段路徑共2次open、3次read、4次write、1次fsync與2次close，即12次API操作。表的核心不是固定次數，而是成功的short write會改變後續buffer位置；只在每次read之後呼叫一次write會漏掉3976 bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1　複製迴圈的一般不變量與失敗界線

設已讀bytes為R、已寫bytes為W、目前buffer待寫區間為[j,n)。迴圈維持W≤R，且buffer[j:n]等於來源尚未寫入目的的連續片段。每次正回傳k≤n−j後，W增加k、j增加k，序列不變；只有j=n才能覆寫buffer並讀下一段。讀到EOF且沒有待寫bytes才可推出W=R=6000。

若read/write在零進度前被signal中斷，依介面處理EINTR並重試；若已返回正bytes，先前進進度，不能整段重寫。未預期的write=0必須避免無限迴圈並回報無進度。ENOSPC或EIO發生時只能聲稱已有部分輸出，不能回成功。

成功write不等於crash後持久。固定本機filesystem/device遵守flush契約時，成功fsync目的可確認檔案資料與必要metadata；新建pathname的directory entry另需同步所在directory。這仍不是「整個copy具atomic replace」：失敗時目的可能部分可見。若需求是替換舊版本，需另設同directory暫存檔、資料同步、rename與directory同步的完整協議，而不是僅加一次close。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 使用者與作業系統介面

- command interpreter 解讀命令並啟動程式，shell 是常見實例；GUI 把操作轉成程式事件。兩者都只是作業系統介面的前端。
- 介面工具若擁有高權限，也會成為攻擊面。命令注入與錯誤檔案覆寫需要由介面層和核心層共同控制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

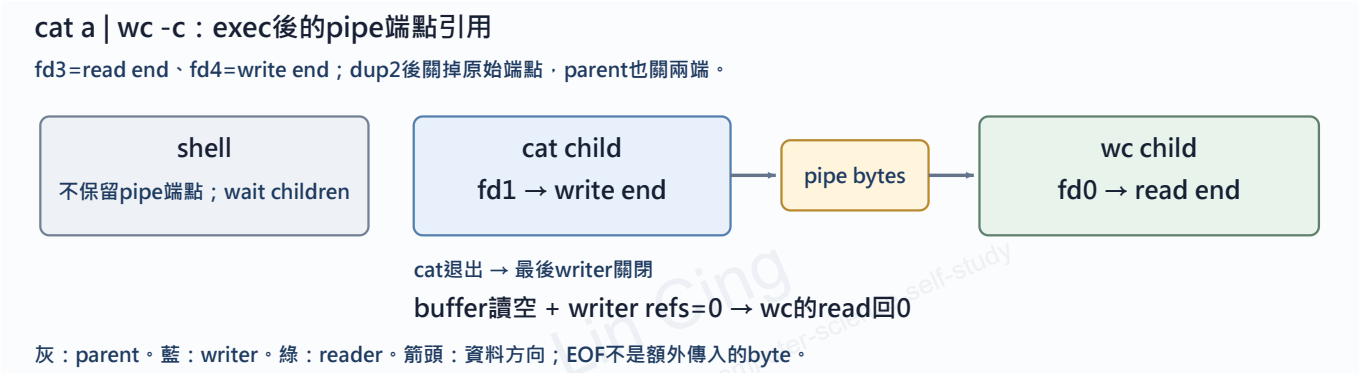
2.2 Shell、GUI與kernel各解讀哪一層

Command interpreter將文字或script轉成程序與I/O連接；GUI將滑鼠/觸控事件轉成應用操作。兩者可以使用相同kernel服務。`cd`若只在child中執行，不會改變parent shell的cwd，所以常是shell builtin；`cat`等外部工具則可由child exec。此差異來自process資源語意，不是「內建命令必然在kernel」。

固定POSIX-style命令`cat a | wc -c`，不含alias、job control或額外子shell。Shell解析兩個command與一個pipe，先建立pipe，再fork兩個child；cat把stdout連到write end，wc把stdin連到read end。原shell仍保留自身program image並wait兩個children；兩次exec替換child的image而非另生兩個process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 Shell、GUI與kernel各解讀哪一層 (續)



2.2 端點引用決定EOF：不能只畫一條管線

假設pipe()回fd3=read end、fd4=write end，fork後三者暫時都持有這兩個descriptor。cat執行dup2(4,1)，wc執行dup2(3,0)，然後各自關掉不再需要的原始fd3/fd4；shell也關掉兩端。

process	exec後留下的pipe引用	關閉動作	為何必要
shell	無	close(3)、close(4)	避免parent自己讓writer一直存在
cat	fd1→write end	關fd3與原fd4	reader消失時不被自己的read引用掩蓋
wc	fd0→read end	關原fd3與fd4	避免wc自己持有write end而永遠等不到EOF

cat把a的6000 bytes寫入pipe並結束；wc可能用任意分段read，累加總數6000。只有pipe資料已讀空且所有write-end引用都關閉，下一次read才回0。若shell忘關fd4，cat即使已退出，wc也可能阻塞，shell又在wait wc，形成不必要的僵持。不能用「cat結束」代替引用條件。

2.2 輸出、exit status與權限不由介面外觀決定

cat失敗不保證pipeline最右端wc也失敗。例如cat開a得到ENOENT，未輸入任何bytes便退出非零；wc讀到EOF仍可能輸出0並exit0。Shell必須分別取得兩個wait status；是否把pipeline整體判為失敗取決於其明定規則。這與資料正確性是兩個不同觀測。

GUI的「複製成功」也不能只憑progress bar走完：應等待相應I/O/error回報與宣稱的同步契約。把GUI換成CLI不會改變目的檔案的權限檢查、容量或crash一致性。

固定字串參數含空白時，parser必須區分token邊界與參數內容。例如`cat 'a b'`傳入一個pathname；`cat a b`傳兩個。Kernel只收到exec參數陣列，並不重新套用shell的引號/pipe語法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 System calls

- API 是程式設計介面，system call 是進入核心的受控操作。POSIX 或語言函式庫可包裝多個 system call，因此名稱與數量不必一一對應。
- 參數必須從使用者記憶體安全複製，核心要驗證長度、位址與權限，避免任意位址讀寫。

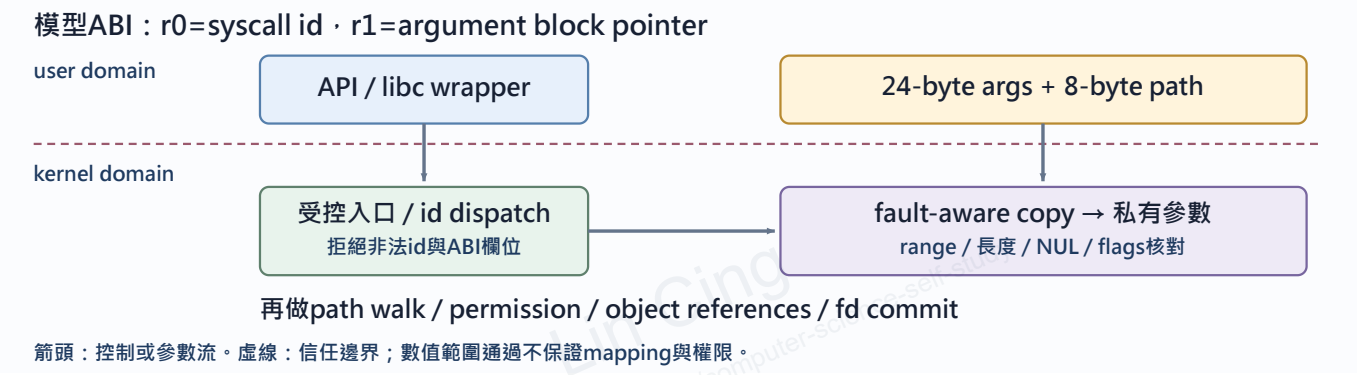
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 API、ABI與system call是三個不同契約

API規定函式名稱、參數型別與可觀察行為；ABI規定machine-level calling convention、syscall number、register/stack layout與返回方式；kernel implementation處理受保護物件。Source能以同一API重新編譯，不表示原binary可在不同ISA/ABI直接執行。

`strlen()`通常可完全在user space；buffered `printf()`可能先留在library buffer，之後一次write輸出多次printf的內容；一個copy API也可組合多個open/read/write。故API呼叫次數不是trap次數。若libc將kernel錯誤碼轉成−1/errno，也應分開描述兩層返回格式。

2.3 API、ABI與system call是三個不同契約 （續）



固定教學ABI：syscall id在r0，r1指向24-byte argument block，三個64-bit欄位依序是path pointer、flags、mode。這是明定的模型ABI，**不是聲稱Linux所有open都用24-byte memory block**；真實平台可用registers傳遞，歷史實作也會變動。

2.3　參數複製、指標驗證與TOCTOU

模型pathname為`/data/x`加NUL，共8 bytes。核心先以受保護user-copy取24-byte固定區塊，再按有界string規則取至多PATH_LIMIT bytes；此例成功取8bytes，總copy32。只有複製到kernel-owned storage後，才對同一份內容解析與使用，避免之後重新讀取user陣列而得到另一條path。

步驟	核對內容	失敗結果
解碼id/ABI	id存在，欄位格式/flags合法	拒絕，不能跳到任意handler地址
複製argument block	user range與實際fault-safe copy	EFAULT或相應模型錯誤，不使用半份參數
複製pathname	有NUL且不超長；來源可存取	EFAULT或ENAMETOOLONG，無fd公布
path walk	每個directory具search權；最後物件權限符合flags	EACCES/ENOENT等
配置與公布fd	穩定引用、回滾未完成配置	成功fd；失敗不洩漏已分配引用

2.3　參數複製、指標驗證與TOCTOU （續）

先查VMA「看似可讀」不是充分條件：另一thread可unmap、page可fault。真正copy操作必須可處理fault，檢查是否完整複製，再只驗證與使用同一份kernel私有副本。私有副本在複製完成後不再被user修改，**不表示複製期間讀到的是某個瞬間的原子snapshot**。

例如user的兩欄位原為(length=4, capacity=4)。Kernel先讀length=4，user依序改length=8、capacity=8，kernel再讀capacity=8，便取得(4,8)；user曾有(4,4)、(8,4)、(8,8)，卻從未有(4,8)。因此必須把私有副本視為任意不可信輸入，重新驗證欄位間關係、長度及權限；不可宣稱一次copy保證來源多欄位一致。若API真正要求來源一致snapshot，需另外提供可強制的同步／版本協議，且不能信任惡意caller會遵守user-space鎖。Filesystem namespace在path walk中的穩定性則另需kernel物件引用與同步。

2.3　範圍與乘法溢位：先證明算式才取記憶體

固定32-bit無號machine word、user地址區間[0,U)，U=0xC0000000；要讀長度n的非空範圍。採`p < U && n <= U-p`，避免先算p+n而溢位。算式通過只表示數值範圍合法，仍須驗證實際mapping/permission並以安全copy處理fault。

p	n	exclusive end（數學整數）	範圍判斷
0xBFFFFF00	0x100	0xC0000000	合法，最後byte是0xBFFFFFFF
0xBFFFFF00	0x101	0xC0000001	非法，跨user邊界
0xFFFFFFF0	0x40	0x100000030	非法；32-bit加法會偽裝成0x30

若count個element每個4bytes，先驗證count≤SIZE_MAX/4；count=0x40000001會使數學乘積0x100000004，32-bit卻變4。只配置4bytes卻循環寫count個element就是越界來源。零長度操作是否允許任意pointer屬個別API契約，本頁判斷刻意限定非空範圍，不能任意套到所有syscalls。

2.4　System call 類型

- 程序控制、檔案管理、裝置管理、資訊維護與通訊是最常見類別。參數可透過暫存器、記憶體區塊或 stack 傳遞。
- 通道式 IPC 以較長生命週期連結程序，訊息式 IPC 以有界資料單元交換。語意不同，正確性條件也不同。

2.4 System call分類應接到物件與狀態

類型	代表請求	受保護物件/狀態	成功不代表什麼
Process control	fork、exec、wait	task、image、exit status	exec不是額外建立process
File management	open/read/write/lseek/close	fd、description、inode	write成功不是持久承諾
Device management	read/write、control request	driver queue、device state	送入queue不是裝置完成
Information maintenance	get/set attributes、time	credentials與system metadata	知道物件ID不是持有權限
Communication	pipe、send/receive、shared memory	endpoint、message、mapping	byte-stream read不是一則訊息
Protection	permissions、credential操作	access policy與principal	authentication不是每次authorization

2.4 System call分類應接到物件與狀態 （續）

分類允許重疊：socket read同時具有I/O、通訊和保護面向；不能只因函式被放在「file」欄便假定所有fd都是regular file。相同read介面可對檔案、pipe或terminal有不同blocking/EOF/seek語意。

2.4 回傳值是狀態機的輸出，不是裝飾

固定regular-file read，count=4096且無並行修改。從offset0開始：read回3000→offset3000；下一次回1096→offset4096；再回0表示該時刻EOF。第一次short read不等於永久EOF；對pipe更可能只是目前可用bytes少於請求。

再看signal操作：模型kill(pid=100,sig=9)的兩個32-bit整數共8bytes，表示方式不等於實際copy成本。核心依pid namespace找對象並查caller權限及signal有效性；目標不存在回ESRCH，無權回EPERM。成功表示signal送達/排入相應處理機制，不表示caller已觀察到目標完全退出與所有資源回收；需要wait等生命週期同步。

若要求多步atomic交易，必須查哪些步驟共用commit point；兩次各自成功的syscall不自動變成一個atomic操作。保護、blocking、visibility、durability與termination是不同保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 System programs

- Compiler、shell、loader、檔案與通訊工具組成user-space工作環境，透過kernel primitives取得資源；system program名稱不授予kernel mode。
- 工具鏈每段的輸入/輸出與exit status都要正確銜接，才能防止舊object/image被誤當作本次成功結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5　從source到running process：system programs做了哪些事

固定C程式hello.c，只需一個main與libc輸出。工具鏈路徑為preprocess→compile→assemble→link→可執行image；動態連結時，exec還需要interpreter/loader與相容libraries。Compiler與linker是user programs；它們為讀寫檔案、配置記憶體、建立子程序使用kernel服務，並不因生成機器碼而在kernel mode執行。

階段	主要輸入/輸出	可獨立發生的失敗
Preprocessor	source/headers→展開後tokens	header缺漏、macro條件錯誤
Compiler/assembler	tokens→assembly→object	syntax/type error、target ISA不合
Linker	objects/libraries→image	unresolved symbol、ABI不合
Exec/loader	image/interpreter→process image	權限/格式/loader缺漏
執行	entrypoint→API/syscalls	runtime fault、I/O錯誤

檔案管理工具、status工具、programming support、program loading、communication與background service屬不同用途；是否長期daemon化，不改變其仍受CPU模式及核心介面保護的事實。

2.5　Shell串接的錯誤傳播不能被分號吃掉

固定三個工具parser、compiler、linker，成功exit0、失敗非零。`parser && compiler && linker`只在前項成功才做下一項；`parser ; compiler ; linker`無條件繼續，最後狀態可能只反映linker。為避免舊object被誤用，build系統還需明定輸出版本與依賴，不可僅判斷檔案存在。

測試	parser	compiler	linker	合理結論
A：全部成功	0	0	0	image產生，仍未證明能run
B：parser語法失敗，&&	2	未執行	未執行	build失敗，不發布舊image
C：compiler失敗，;	0	1	0（吃到舊object）	最後0掩蓋前項失敗

Source-level API相容、binary ABI相容與runtime dependencies齊全都要分別成立。只看到link成功或某個digest一致，不能推論整個target環境可開機或應用無錯。

2.6　設計與實作

- Mechanism提供受控入口、timer與context保存；policy決定選誰、何時、配多少。相同機制可支援不同quantum與選取規則。
- 使用模型數字須明列切換成本、arrival與burst；未量測的實機throughput/p99不能填造。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6　Mechanism與policy：用同一個timer做兩種選擇

Mechanism提供可安全執行的操作，policy選擇何時/對誰/多少。Timer interrupt、保存/還原執行狀態、ready queue操作是排程mechanisms；quantum長度與挑選順序是policy。分離不表示沒有相互限制：timer精度、nonpreemptible區段與切換成本會限制policy能達成的延遲。

固定單CPU，A/B/C皆在t=0就緒，CPU bursts5/3/1ms，round-robin初序A,B,C，無I/O、切換成本0。相同mechanism，僅改quantum。

q	完整CPU區間	completion(A,B,C)	first response(A,B,C)
1ms	A0–1,B1–2,C2–3,A3–4,B4–5,A5–6,B6–7,A7–8,A8–9	9,7,3	0,1,2
4ms	A0–4,B4–7,C7–8,A8–9	9,7,8	0,4,7

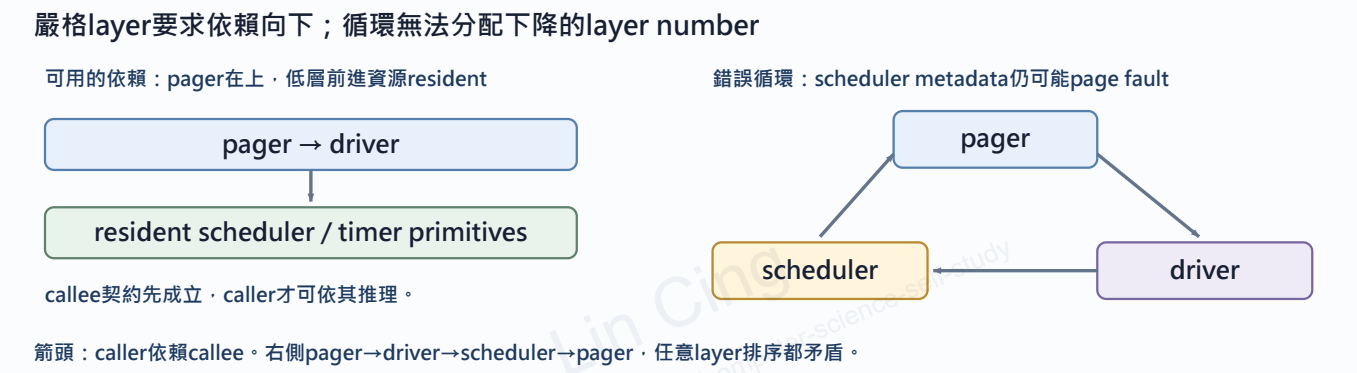
兩者總useful CPU time都9ms；q1平均response=1ms，q4為11/3ms。q小提高反應性，不代表實機無代價：若每次真正task switch有成本s，q1有7次task switches、q4有3次；q1最後A→A不算task switch。固定成本會改變wall time，必須重算，不能把上表零成本時間直接搬用。

2.6　分層證明有前提：循環依賴破壞歸納

嚴格layered設計要求每個服務僅依賴更低層已給定的契約。若第0層滿足C0，第i層在C0…C(i−1)成立下可證Ci，則可由歸納建立上層性質。實際測試通過不是無條件「下層絕不可能出錯」；契約、前提或測試覆蓋錯誤仍會使推論失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6　分層證明有前提：循環依賴破壞歸納 （續）



例：VM pager依賴disk driver，disk driver等待時依賴scheduler；若scheduler的必要metadata又可能page fault而依賴pager，就形成pager→driver→scheduler→pager。三者無法同時滿足嚴格下降的layer number。可把scheduler必要資料與page-fault/I/O前進所需核心資源保留resident，拆出不依賴paging的底層原語；不是把循環箭頭藏起來就完成設計。

2.6 Implementation語言與可測量的設計目標

高階語言可改善可讀性、移植與compiler最佳化，但不能自動保證memory safety、atomicity或hardware ordering；C的pointer錯誤仍可破壞kernel。Assembly常留於入口、context save、特權指令等machine-specific邊界；應用同一library API不會自動消除這些ABI差異。

固定queue容量1024、相同預先產生的操作序列與machine資源。候選A無lost element；候選B雖較快卻在一個已記錄交錯把已enqueue元素遺失，B違反正確性契約而不可取。這是邏輯排除，不必等平均throughput超過門檻才決定。

效能目標須附workload與分母：單一request的CPU time、端到端latency與每秒完成數不同；p99以sorted observations的明定quantile計算。零切換成本的排程表只回答所列模型；實機比較還需量測切換、cache與等待成本，不能把不同分母的時間直接相除。

2.7 作業系統結構

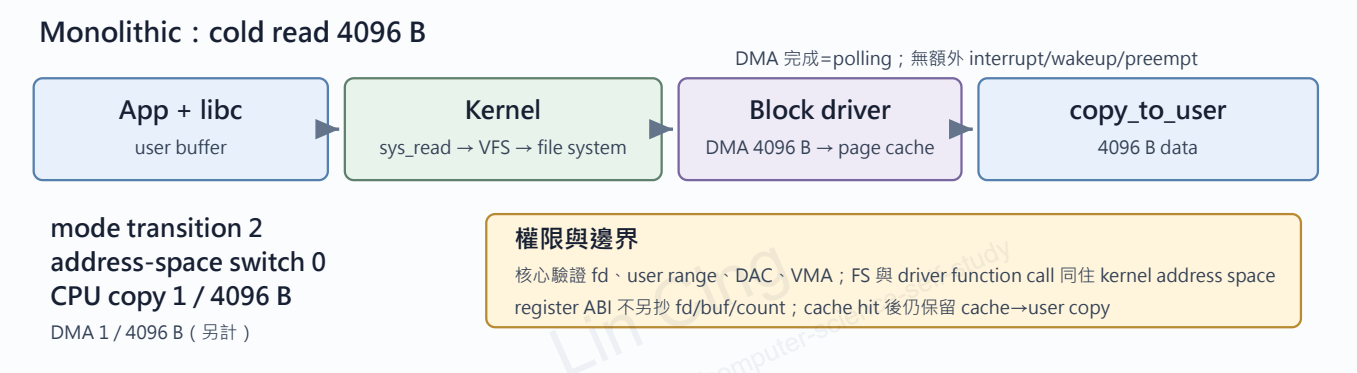
- monolithic 核心把 VFS、file system 與 block driver 放在同一個核心執行環境；system call 進入核心後，各服務以 function call 串接。效能代價低，但 driver 或 service 的缺陷可影響整個核心。
- microkernel 把 file server 與 driver 留在 user-mode processes，由 kernel 提供受檢查的 IPC、排程與基本記憶體管理。每個 server 有獨立 address space，故障可被隔離，代價是額外 mode/address-space transitions 與訊息複製。
- 模型比較必須同時固定 cache 狀態、訊息大小、server/driver 部署與 copy strategy。只換架構名稱而改變 workload，無法比較 mode switch、context switch 或資料複製成本。

2.7　固定檔案讀取：monolithic 與 microkernel 的完整路徑

比較設定固定為 4 KiB cold-cache read：`read(fd, user_buf, 4096)`，檔案為 `/srv/data.bin`，offset 0。ABI 明定 `fd`、`user_buf` 與 `count` 由 registers 傳遞，不把參數另行計算成 `copy_from_user`。monolithic 模型把 VFS、file system 與 block driver 放在 kernel；microkernel 模型把 file server 和 driver 放在兩個獨立 user address spaces。microkernel 的 kernel mapping 位於當前 user address space；IPC 的 sender→kernel→receiver two-copy strategy 與 user-address-space handoff 分開計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.7　固定檔案讀取：monolithic 與 microkernel 的完整路徑 （續）



monolithic 的 function call 路徑如下。應用程式先以 libc 發出 read；CPU 從 user mode 進入 kernel mode。核心檢查 descriptor、access rights、user buffer 與 current offset，再依序呼叫 VFS、file system 與 block driver。driver 用 DMA 把 disk page 放入 page cache，核心再把 4096 bytes 複製到 user buffer，最後返回 user mode。

2.7 固定檔案讀取：monolithic 與 microkernel 的完整路徑 (續)

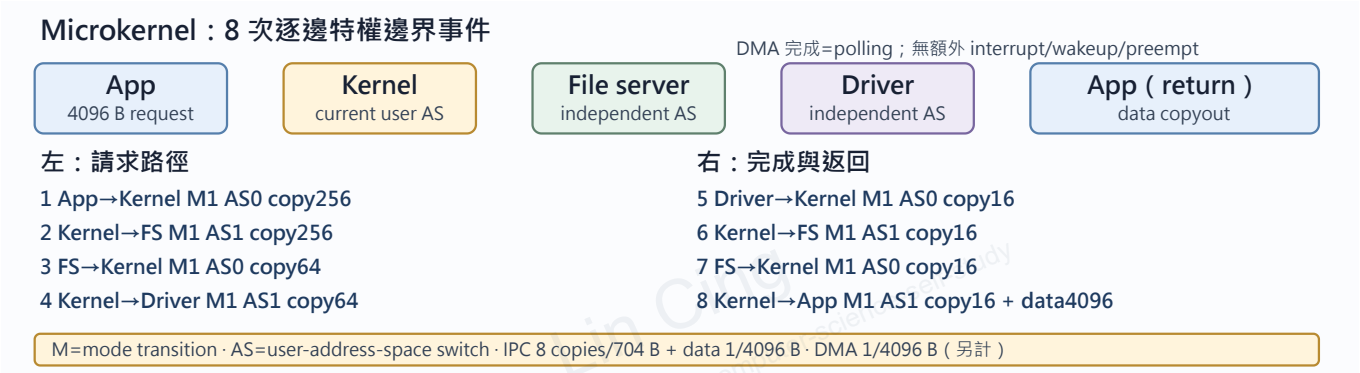
事件	執行位置	邊界或呼叫	成本計數
`libc read`	App user space	ABI 呼叫	0
`sys_read`	Kernel	register arguments；user→kernel mode	mode +1
`vfs_read`、file-system read、driver read	Kernel	同 address space function calls	address-space +0
disk→page cache	Driver／DMA	4096-byte DMA	CPU copy +0
page cache→user buffer	Kernel	4096-byte copy	CPU copy +1
return 4096	App user space	kernel→user mode	mode +1

2.7 固定檔案讀取：monolithic 與 microkernel 的完整路徑 (續)

register ABI 不再增加參數 copy。因此總數是 **2 次 mode transitions、0 次 address-space switches、1 次 4096-byte CPU copy 與 1 次 4096-byte DMA**。改成 cache hit 後，driver 與 DMA 不出現，但 cache→user 的 copy 仍在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.7 microkernel：IPC、server、driver 與計數



microkernel 模型中，application、file server 與 driver 是三個 user-mode processes。Kernel mapping 隨當前 process 的 address space 生效，所以 `App→Kernel` 或 `FS→Kernel` 只做 user/kernel 的 mode transition，不另換 address space；`Kernel→FS`、`Kernel→Driver` 等交接到另一 user process 才算 AS switch。

2.7 microkernel：IPC、server、driver 與計數 （續）

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
1	App→Kernel	register args；`SYS_IPC_SEND(read_req)`	256 的 sender→kernel copy	1	0	1

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
2	Kernel→File server	deliver read request	256 的 kernel→receiver copy	1	1	1

2.7 microkernel：IPC、server、driver 與計數 （續）

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
3	File server→Kernel	`SYS_IPC_SEND(block_req)`	64 的 sender→kernel copy	1	0	1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.7 microkernel：IPC、server、driver 與計數 （續）

步驟	特權邊界事件	ABI／IPC	控制訊息 bytes	mode transitions	address-space switches	CPU copies
4	Kernel→Driver	deliver block request	64 的 kernel→receiver copy	1	1	1
5	Driver→Kernel	completion IPC send	16 的 sender→kernel copy	1	0	1
6	Kernel→File server	deliver completion	16 的 kernel→receiver copy	1	1	1
7	File server→Kernel	reply IPC send	16 的 sender→kernel copy	1	0	1
8	Kernel→App	deliver reply；copy page cache	16 的 kernel→receiver copy＋4096 資料	1	1	2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.7 microkernel：IPC、server、driver 與計數 （續）

總數是 **8 次 mode transitions、4 次 user-address-space switches、9 次 CPU copies**。四則 control messages 的大小為 256、64、16、16 bytes；two-copy 後是八次 copy、704 bytes。最後再加 4096-byte 資料 copy，所以總共 copied bytes=4800。DMA 另有 1 次 4096 bytes，不混入 CPU copies。AS switches 對應 App→FS、FS→Driver、Driver→FS、FS→App，皆由 kernel handoff 到不同 user address space。

monolithic 的 function call 不會自動變成 zero-copy；microkernel 的 IPC 也不必然每次執行 eight transitions。本頁比較的是一個完整、可重算的成對模型。若 server 與 driver 合併為同一 process、kernel 使用獨立 private address space，或改用 shared-memory/one-copy IPC，就必須重新列 ABI、AS 配置與計數，不能沿用這組常數。

2.7 DMA 邊界與 IPC 成本的敏感性

前兩頁的 2/0 與 8/4 只涵蓋 request chain 內明列的服務交接。DMA 完成採 **polling**；模型中沒有其他 runnable work，也不發生 timer preemption 或 completion interrupt 喚醒。若改採 interrupt、阻塞等待或讓其他 process 在等待期間執行，就要另列 interrupt entry/exit、wakeup 與 scheduler switch；它們不包含在這組基準數字內。

2.7 DMA 邊界與 IPC 成本的敏感性 (續)

變更	mode transitions	address-space switches	CPU copies	copied bytes
基準：monolithic，current-AS kernel mapping	2	0	1	4096
基準：microkernel，256/64/16/16 messages	8	4	9	4800
microkernel 的 request message 由 256 改為 128 bytes	8	4	9	4544
file server 與 driver 合併為同一 process	4	2	5	4640
private-kernel-AS 模型，monolithic	2	2	1	4096
private-kernel-AS 模型，microkernel	8	8	9	4800

2.7 DMA 邊界與 IPC 成本的敏感性 (續)

128-byte request 的 two-copy 控制資料是 `2 × 128 + 2 × 64 + 4 × 16 = 448 bytes`，再加 4096-byte 資料，得到 4544 bytes。合併 server 與 driver 後只剩 request 256 bytes 與 reply 16 bytes 兩則 IPC；two-copy 為 544 bytes，加資料後是 4640 bytes。這些列改變的是已標示的模型參數，不代表任何真實 OS 的固定 syscall 成本。

private-kernel-AS 模型也必須套用到 monolithic：每次 user/kernel mode transition 都伴隨 address-space switch，所以 monolithic 由 0 次 AS switch 變成 2 次，microkernel 由 4 次變成 8 次。頁面只比較模型，不把 polling、zero-copy 或 fast-path 條件外推為所有 read 的一般成本。

2.7 結構比較的四個獨立維度

結構	呼叫/依賴	權限與address space	動態擴充	故障與契約
Monolithic	多數服務kernel function calls	多數共享kernel privilege	可有loadable modules	bad pointer可污染共用核心
Strict layers	只依賴較低層介面	layer不必是硬體保護domain	可靜態或動態	契約有助逐層分析，但未自動隔離記憶體
Microkernel	最小核心原語+跨server IPC	多個受隔離user domains	server可分別替換	core較小；parser/權限/恢復仍需正確
Modular kernel	經已定義module interfaces連結	module仍可能共享kernel privilege	runtime load/unload	可維護性不等於fault isolation
Hybrid	依各子系統選部署	需逐項標domain	可混合module/server	名稱不足以推論成本或安全

2.7 結構比較的四個獨立維度 (續)

Layer是dependency概念，module是組織/連結概念，protection domain是存取控制概念。把程式拆成不同檔案或module，不會自動建立user/kernel邊界；把所有軟體畫成上下方塊，也不代表依賴真的無環。

2.7　隔離可降低破壞範圍，不會自動完成恢復

固定file server crash，client已送出「append record R」，卻未收到reply。若server未commit就crash，retry才可補做；若已commit但reply遺失，無條件retry會append兩次。隔離只防止server任意寫kernel memory，沒有回答client如何辨別這兩種history。

可加入request ID與可恢復的dedup/result紀錄，使同一ID重試取回相同結果；若紀錄未持久或與append不同transaction，crash仍可能留下不一致。權限應限制server可控制的device/memory/handles；若錯誤server仍有整顆disk任意寫入權，它能破壞磁碟資料，即使無法改寫kernel。

動態module卸載也要先禁止新入口，再等既有references、DMA與callbacks結束；僅把symbol從table移除不足以保證舊function pointer不再執行。這與user-space server restart是不同生命週期問題。

2.7　教材歷史系統：混合元件不等於相同邊界

以本書出版時的Mac OS X案例：Aqua/Cocoa在上層，kernel environment包含Mach、BSD與I/O Kit/extension。圖中列出Mach並不代表BSD服務一律在獨立user process；判定mode/AS成本仍要查實際部署。iOS以Cocoa Touch、Media Services、Core Services、Core OS表達分層服務。

同一歷史Android案例為applications→application framework→libraries與Dalvik runtime→Linux kernel。Dalvik是該教材的runtime，不把它當現今所有Android的固定實作；Linux kernel提供process/memory/device等服務，不執行Java API的所有高層語意。

三者要比較的是「哪一層提供哪個contract、在哪個domain、跨哪條邊界」。上層framework名稱與同一張stack圖不能替代前述逐事件read trace；也不能把歷史架構直接外推至新版本。

2.8　作業系統除錯

- kernel panic、core dump、log 與 tracing 提供不同可見度。核心除錯工具必須避免改變時序，否則 race condition 可能被觀測行為掩蓋。
- 效能分析要區分 CPU、等待、快取、鎖與 I/O 延遲。單一平均數無法定位長尾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.8　Failure analysis：證據反映哪個時刻

User core dump保存失敗process的部分執行/記憶體狀態，kernel crash dump則保存核心故障現場；log是程式選擇輸出的事件，trace可觀察呼叫、排程、I/O等時間順序。它們互補，並非任何一種都能還原完整歷史。

證據	可以回答	不能只憑它斷言
fault PC、registers、stack	哪個指令/呼叫鏈失敗	第一個破壞記憶體的位置必在此處
heap/object snapshot	crash當時物件值	值何時被哪個thread改壞
timestamped trace	已記錄事件先後與時間分解	未開probe的事件沒有發生
CPU sampling	哪些stack累計CPU活動	I/O等待最多的request一定被採到

若filesystem本身壞了，要求故障kernel先透過同一filesystem寫完整dump可能再次失敗。需預留可靠dump路徑/空間與重啟後擷取方法；蒐集失敗不能當作「沒有故障」的證據。

2.8　10ms延遲：CPU、blocked與ready分開加總

固定單一request，clock是可比較的monotonic時間，事件無遺失。t0進handler，t1送I/O並阻塞，t8裝置完成而task變ready，t9得到CPU，t10回傳。CPU執行[0,1)+[9,10)=2ms，blocked[1,8)=7ms，ready queue等[8,9)=1ms，end-to-end=10ms。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.8　10ms延遲：CPU、blocked與ready分開加總 （續）

同一request的10ms：CPU 2 + blocked 7 + ready等待 1

橫軸比例1ms=100px；區間左閉右開。



綠：CPU執行。黃：blocked。紫：已可跑但尚未選中。完成中斷不代表立即回到user。

Lin Cing

把7ms都叫「磁碟CPU時間」會誤判；把t8完成當成t8返回會漏掉ready delay。若probe開銷改變排程，量測系統已不同，需對照未插樁時的輸出與可觀察時間，而不是假設debug build零干擾。

八個request latency為1,1,1,1,1,1,1,100ms，mean=107/8=13.375ms；採nearest-rank p99，rank=ceil(0.99×8)=8，p99=100ms。小樣本p99只是此樣本的order statistic，不構成穩定尾端機率估計。

2.8　觀測會改交錯：race必須以同步條件排除

初始x=0，兩個threads都做read x、加1、write x，沒有鎖。合法交錯T1:r0→T2:r0→T2:w1→T1:w1，最後x=1；serial順序則為2。加入printf可能把原本重疊改成serial，輸出2不等於data race已修好。

正確性要求的不是「debug跑了十次沒錯」，而是所有容許交錯都保證兩次increment不可分割，例如兩次用同一mutex保護完整read-modify-write，或使用滿足需求的atomic RMW。Lock若只保護write而不保護read，兩者仍可先讀0再各寫1，反例未消失。

動態probe應保留事件ID、時間域、drop計數及必要object identity；每CPU timestamp的排序假設也要明示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9　作業系統產生

- SYSGEN將CPU/ABI、RAM、device、root與選用功能轉成compile/link/runtime配置。必要driver的依賴必須在用到它之前可取得。
- 相容性與依賴閉包可做preflight；檢查通過不代表hardware、driver邏輯與真實boot皆已驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9　SYSGEN：把硬體配置接到可滿足的依賴

System generation可以在compile time裁切source，在link time選預編譯元件，或runtime依table/偵測結果選功能。三者取捨是映像大小、可移植性、硬體變更成本與早期可用性，不能僅以「有沒有重新編譯kernel」二分。

配置項	固定機器需求	缺漏的可觀察後果
CPU/ABI	kernel與使用者image為相容ISA/ABI	未支援指令/exec格式不合
RAM與保留區	使用可用實體範圍，避開device/firmware reserved	覆寫保留資料或非法映射
Boot storage	controller driver能讀root所在device	看不見root device
Filesystem	root filesystem driver早期可用	device存在仍無法mount
Root/init	正確root識別、可執行init及其loader	mount成功卻無法啟動userspace

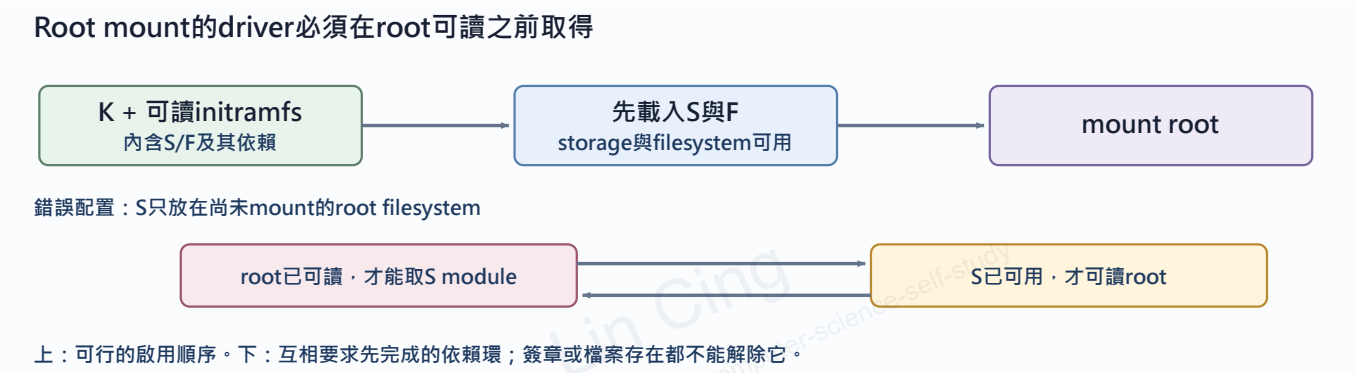
Runtime hardware discovery需要合適driver，不能把「bus上找得到device」當成已有可用I/O。模組檔案存在也不代表kernel版本/ABI可接受，更不代表在需要它之前能讀到它。

2.9　Root device的driver不能只放在尚未mount的root

固定root在storage S，使用filesystem F；K已內建其餘必要依賴。以B表示built-in、I表示initramfs可載入的相容module、R表示僅放在root。早期需求是S與F在root mount之前皆可用。

S位置	F位置	initramfs內容	能否到root mount
B	B	不需S/F modules	能
I	I	S、F及各自依賴皆有	能，先載入再mount
R	B	缺S	不能；須先讀root才取得讀root的driver
B	R	缺F	不能；有device但不認root格式
I	I	只有F，S缺漏	不能；必要依賴未閉合

2.9 Root device的driver不能只放在尚未mount的root （續）



這是dependency reachability問題：由built-ins與initramfs可讀元件起始，反覆加入「所有prerequisites都已可用」的元件；若mount-root永遠未加入，就有缺依賴或循環。依賴閉包成立只排除這一類缺件，仍不能證明hardware、driver邏輯或root內容都正確。

2.10 System boot

- Firmware/loader/kernel/root/init是責任交接；bootstrap角色與特定BIOS/UEFI細節分開。
- 明定Linuxv3.2 start_kernel→rest_init→kernel_init與userspace init路徑；抽象verified-boot政策另列，避免把簽章或降權寫成所有系統既有保證。

2.10　從reset到init：每次交接都有最低前提

固定教學機器的firmware可找到bootloader；loader能載入kernel與initramfs並提供memory map/command line。這是角色模型，不把BIOS單扇區流程與UEFI流程硬合成同一個實作。

階段	主要輸入	產生的下一階段前提
Reset/firmware	固定reset入口、硬體配置	基本可用CPU/RAM、可執行loader
Loader	boot設定、kernel/initramfs images	image載入到正確區域，參數交接
Early kernel	hardware/boot資訊	page tables、interrupt、scheduler、memory基本設施
Root setup	storage/filesystem drivers	root可mount，userspace檔案可讀
Userspace init	init image、interpreter、libraries	建立服務/登入環境；權限依各服務policy

Linuxv3.2 `start_kernel()`完成早期初始化後，`rest_init()`建立執行kernel_init的task；不是`start_kernel()`直接跳去執行任意shell。`kernel_init()`完成initcalls/root準備與後續交接，最後嘗試執行相應init。initramfs有可執行`/init`時可先走它；沒有合適早期init時需要準備root namespace。錯誤位置不同，留下的診斷也不同。

2.10　Boot失敗的狀態比「不能開機」更有資訊

固定可驗證階段F→L→K→R→I：firmware找到loader、loader交接kernel、kernel可讀root、root可讀init、init成功exec。令gate只在其輸入成立時前進。

故障注入	最後成功階段	首個無法完成的動作	不應歸咎
loader找不到kernel	L	載入K	不能先怪init服務
缺S driver	K	讀root device	不只是root pathname拼錯
F driver缺漏	K	mount已存在的root device	不等於disk不存在
init loader缺漏	R	exec init	不等於kernel沒起來

這張table是可追蹤的role model，不宣稱所有Linux錯誤都以同一exit code回傳到firmware。Kernel panic、loader錯誤畫面、userspace非零退出是不同失敗通路，無法用「任一stage回1就停止」描述完整真實開機。

2.10 Verified boot、measured boot與相容性分開

若採verified-boot設計，可信前階段驗證下一階段的簽章/授權policy，並在失敗時拒絕執行；只計算或記錄digest屬量測，沒有自動阻擋。下一階段包含哪些image/config/initramfs，須在policy中明列，不能只驗kernel檔名。

即使所有簽章合法，舊且已撤銷的版本仍可能不應接受；需要版本/撤銷policy。被授權但本身有漏洞的image也不因簽章正確而無漏洞。反過來，ABI匹配與driver依賴閉合只能支持相容性，不證來源可信。

前一頁Linuxv3.2的啟動控制流與這個抽象信任鏈分開：不宣稱v3.2 mainline自帶所有此處政策，也不假設init立即移除所有capabilities。服務啟動後的權限分配需依具體工作與安全設計決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.11 同一個read跨越哪些契約

以2.7的4096-byte cold read收束：應用API提供fd/buffer/count；ABI決定registers；受控入口建立kernel執行上下文；核心驗證fd及buffer，VFS/filesystem依描述物件找到資料；driver/DMA完成cache填入，再copy到user buffer，回4096。Function call、mode transition、AS handoff、CPU copy、DMA是不同事件，不能重複或互換計數。

把實作改成microkernel，API可以維持同樣read契約，內部卻增加server/driver IPC。把shell換成GUI，前端改變但read保護規則不因此消失。把kernel拆成modules，載入方式改變但未自動縮小每個module的kernel privilege。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.11　成功、故障與恢復要接到真正的完成點

讀回4096表示此次bytes可供應用使用，不表示後續write已完成或storage已持久；新pathname的持久化也不同於檔案資料同步。對pipe，讀到EOF取決於buffer空且所有writers消失；對server request，收不到reply不能判定操作未執行。

設計時用契約與不變量連起服務；除錯時用物件ID與時間線找第一個被破壞的條件；SYSGEN/boot用依賴閉包確認所需機制在交接前可用。這些推論依賴的前提都能落在本章的buffer進度、fd引用、地址範圍、事件表或module依賴上，而不是由架構名稱決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

- *Operating System Concepts*, 9th Edition, Chapter2，印刷頁55–93；§2.3 Fig2.6的API/syscall入口與§2.7 Fig2.13的layer依賴為本章抽象定位。
- Linux man-pages write(2)、fsync(2)：short write進度與file/directory持久化對象；本章另明定固定copy工作負載。
- pipe(7)：byte-stream、端點引用、EOF與blocking條件。
- Linuxv3.2 init/main.c：start_kernel、rest_init、kernel_init與userspace init交接；信任鏈段另為明定設計模型。
- 本章24-byte argument block、RR工作負載與root依賴表均是明定教學模型；歷史Mac OS X/iOS/Dalvik案例對應教材版本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study